Методические указания к зачетному заданию

По итогам курса обучения предлагается разработать игровую программу.

Темы заданий на выбор:

1. Игра «Угадай Число».
2. Игра «Бросание игрального кубика (игральные кости)».
3. Игра «Угадай слово (принцип *Поля чудес*)».

Можно выбрать свою тему, воспользовавшись примерами описаний предложенных игр (в книге [2] в пункте 7.2 описана игра «Кубик», в пункте 7.3 описана игра «Отгадай число»). Конечно, приветствуется ваши дополнения к сценариям игр.

В этом руководстве будет рассмотрена игра «Бросание игрального кубика», в которой реализуется механизм имитации случайных событий, который используется практически во всех игровых сценариях.

Указание 1. Спецификация требований к программе

На этом этапе требуется уяснить принципы определения функциональных требований к ПО и закрепить навыки их разработки.

Это указание выполняется по указаниям к практическому занятию 1.

Указание 2. Разработка прототипа программы

Для разработки прототипа программы изучите пункт *7.2. Игра «Кубик»* или пункт *7.3. Игра «Отгадай число»* в книге [2].

Далее будет рассматриваться вариант игры «Кубик», названный «Бросание игрального кубика (игральные кости)».

Допустим, принято решение, что функциональное требование «*Система должна реализовать бросание кубика двумя игроками и определять победителя*» является основным, т.е. ядром программы. В этом случае требуется реализовать следующие действия по разработке программы:

1. Создайте программу и добавьте следующий код, реализующий ядро игры (обратите внимание на использование модулей **random** и **time**):

```
from random import randint
import time
```

```
#Ввод имен играющих
gamer1 = input('Введите имя 1-го играющего ')
gamer2 = input('Введите имя 2-го играющего ')

#Моделирование бросания кубика первым играющим
print('Кубик бросает', gamer1)
time.sleep(2)
n1 = randint(1, 6)
print('Выпало:', n1)

#Моделирование бросания кубика вторым играющим
print('Кубик бросает', gamer2)
time.sleep(2)
n2 = randint(1, 6)
print('Выпало:', n2)
```

2. Реализуйте определение победителя игры. Для этого используйте оператор **if-elif-else** для реализации выбора из нескольких альтернатив.

Возможное решение:

```
#Определение результата (3 возможных варианта)
if n1 > n2:
    print('Выиграл', gamer1)
elif n1 < n2:
    print('Выиграл', gamer2)
else:
    print('Ничья')
```

3. Запустите игру и проверьте работу программы.

Можно считать, что прототип программы создан.

Указание 3а. Наращивание функционала программы. Циклы.

Дальнейшие действия: постепенно наращивать функционал прототипа – реализовывать остальные функциональные требования программы.

Например, допустим, что есть такое функциональное требование: *Система должна обеспечить возможность сыграть игру, состоящую из нескольких ‘партий-сеансов’.*

В этом случае требуется реализовать повторение с помощью циклов. Подумайте какой из известных циклических конструкций подойдет для реализации указанного функционального требования.

Добавьте в разработанную программу возможность бросать кубик игрокам несколько раз и по итогам этих бросков определять победителя (например, по количеству побед в сеансах или по общей сумме очков).

Предположим, что игроки заранее договорились о числе партий (бросков). В таком случае подходящим циклом будет цикл **for..in**.

1. Добавьте в программу цикл **for..in**, реализующий 5 партий (бросков):

```
for i in range(5):
    #Моделирование бросания кубика первым играющим
    ...
```

Обрате внимание на указание отступов (4 пробела) для тела цикла.

2. Добавьте реализацию определения победителя. Например, с помощью подсчета количества побед каждым игроком, или суммированием очков.
3. После цикла добавьте оператор ветвления для определения итогового победителя.

Посмотрите возможный вариант решения (обратите внимание на код, реализующий новую функциональность):

```
from random import randint
import time

#Ввод имен играющих
gamer1 = input('Введите имя 1-го играющего ')
gamer2 = input('Введите имя 2-го играющего ')
k1 = k2 = 0    # количество побед каждого игрока

for i in range(5):
    #Моделирование бросания кубика первым играющим
    print('Кубик бросает', gamer1)
    time.sleep(2)
    n1 = randint(1, 6)
    print('Выпало:', n1)

    #Моделирование бросания кубика вторым играющим
    print('Кубик бросает', gamer2)
    time.sleep(2)
    n2 = randint(1, 6)
    print('Выпало:', n2)

    #Определение результата (3 возможных варианта)
    if n1 > n2:
        print('В раунде выиграл', gamer1)
```

```
        k1 += 1
    elif n1 < n2:
        print('В раунде выиграл', gamer2)
        k2 += 1
    else:
        print('Ничья')

# определение итогового победителя
if k1 > k2:
        print('Выиграл', gamer1, "число побед: ", k1)
elif k1 < k2:
        print('Выиграл', gamer2, "число побед: ", k2)
else:
        print('Ничья')
```

Указание 3б. Наращивание функционала программы. Списки и словари.

Допустим, что есть такое функциональное требование: *Система должна обеспечить возможность после окончания игры посмотреть заработанные очки игроков в каждом сеансе*.

В этом случае требуется реализовать сначала запись, а потом и вывод получаемых очков в каждом сеансе в структуру (набор) данных. Подумайте какой из известных структурных данных подойдет для реализации указанного функционального требования.

Использование списков

Добавьте в разработанную программу возможность в каждом сеансе записать результат броска в список игрока, а в конце игры, можно после объявления победителя вывести на экран полученные очки каждым игроком.

Возможная реализация:

1. В начале программы объявляются два списка:

```
rezult_g1 = []
rezult_g2 = []
```

2. В цикле указывается код добавления (метод append) текущего значения в список для первого игрока:

```
    n1 = randint(1, 6)
    rezult_g1.append(n1)
    print('Выпало:', n1)
```

и для второго:

```
    n2 = randint(1, 6)
    rezult_g2.append(n2)
    print('Выпало:', n2)
```

В конце программы можно после объявления победителя вывести на экран оба списка с помощью стандартной функции:

```
print("Результаты первого игрока: ", rezult_g1)
print("Результаты второго игрока: ", rezult_g2)
```

Применение словаря

Добавьте в разработанную программу модель ‘Игрок’, который описывается фамилией, результатами игры (список бросков из прошлого пункта) и числом очков. Подходящей программной моделью на данном этапе является словарь, предназначенный для хранения сложных данных и представляющий собой неупорядоченную структуру данных, которая позволяет хранить пары «ключ — значение».

Возможная реализация:

1. Перед вводом имен играющих создаются два игрока как словари, первый ключ это имя, второй – список результатов и третий это число очков:

```
'''Представление игрока как переменной словаря'''
gamer1 = {'name':'', 'rezult':[] ,'k': 0}
gamer2 = {'name':'', 'rezult':[], 'k': 0}
```

2. Изменяется ввод имен игроков:

```
#Ввод имен играющих
gamer1['name'] = input('Введите имя 1-го играющего ')
gamer2['name'] = input('Введите имя 2-го играющего ')
gamers = [gamer1, gamer2]
```

В этом случает строки с объявлением `k1, k2, rezult_g1=[]` и `rezult_g2=[]` следует удалить.

3. В цикле изменяется обращение к игроку – по нужному ключу:

```
    #Моделирование бросания кубика первым играющим
    print('Кубик бросает', gamer1)
    time.sleep(2)
    gamer1['rezult'].append(randint(1, 6))
    print('Выпало:', gamer1['rezult'][-1])


    #Моделирование бросания кубика вторым играющим
    print('Кубик бросает', gamer2)
    time.sleep(2)
    gamer2['rezult'].append(randint(1, 6))
```

```
    print('Выпало:', gamer2['rezult'][-1])


    #Определение результата (3 возможных варианта)
    if gamer1['rezult'][-1] > gamer2['rezult'][-1]:
        gamer1['k'] += 1
        print('В раунде выиграл', gamer1)
    elif gamer1['rezult'][-1] < gamer2['rezult'][-1]:
        gamer2['k'] += 1
        print('В раунде выиграл', gamer2)
    else:
        print('Ничья')
```

4. После окончания игровых бросков также следует изменить код обращения к игрокам:

```
if gamer1['k'] > gamer2['k']:
        print('В ИГРЕ выиграл', gamer1, "число побед: ", gamer1['k'])
elif gamer1['k'] < gamer2['k']:
        print('В ИГРЕ выиграл', gamer2, ": ", gamer2['k'])
else:
        print('В ИГРЕ "Ничья"')

print(f"Результаты игрока: {gamer1['name']}: {gamer1['rezult']}")
print(f"Результаты игрока: {gamer2['name']}: {gamer2['rezult']}")
```

Указание 4. Применение функций

Дальнейшие действия: для устранения роста сложности программы следует рассмотреть возможность оформления блоков программного кода в виде функций.

- ✓ В решении рассмотрен вариант без представления игрока как словаря. Внесите соответствующие изменения в код самостоятельно.

Реализация функций и главной программы в одном модуле

Первый кандидат на функцию – это алгоритм определения победителя игры. В указанном выше решении он такой:

```
    if k1 > k2:
            print('Выиграл', gamer1, "число побед: ", k1)
    elif k1 < k2:
            print('Выиграл', gamer2, "число побед: ", k2)
    else:
            print('Ничья')
```

Как этот код переделать в функцию?

1. Ответьте на следующие вопросы:

a. Что может быть входными данными для этой функции?

Ответ: входные данные обеспечивают логику (смысл) алгоритма. В чем смысл этого алгоритма – сравнить число побед каждого игрока, вот значит эти количества побед и являются входными данными (они должны передаваться переменным `k1` и k2)

b. Что может быть выходными данными этой функции?

Ответ: выходными данными являются результат алгоритма – в данном случае это строковое сообщение о победителе.

2. В начале программы сразу после инструкций импорта объявите функцию, принимающую два параметра (имена параметров могут быть любыми, даже совпадать с именами переменных в главной программе) и возвращающую строковое сообщение о победителе:

```
def who_champion(a, b):
    if a > b:
        return ('Выиграл {0}, число побед: {1}'.format(gamer1, a))
    elif a < b:
        return ('Выиграл {0}, число побед: {1}'.format(gamer2, b))
    else:
        return 'Ничья'
```

3. Старый код проверки удалите и вместо него объявите переменную, которой присвойте вызов новой созданной функции, передав ей два параметра – переменные – количество побед каждого игрока:

```
fin = who_champion(k1,k2)
print("Итог игры: ", fin)
```

Функциональность программы не изменилась, но теперь отдельный алгоритм выявления победителя вынесен в отдельную функцию.

Ввод данных о пользователях – процедура или функция?

1. Подумайте, как можно было бы реализовать ввод имен игроков?

Вариант 1. В виде процедуры.

В этом случае нужно объявить переменные – имена игроков как глобальные и процедура изменит их первоначальные значения на вводимые.

Этот вариант является не удачным – настоятельно рекомендуется применять глобальные переменные только в крайних случаях.

Вариант 2. В виде функции.

В этом случае функция должав вернуть введенные в ее теле значения имен игроков и присвоить их другим переменным.

Это вариант является рекомендуемым, но есть проблема – как вернуть несколько значений из функции. Вернуть несколько значений из функции можно по-разному (передавая параметры по ссылке, вернуть массив переменных и т.д.), в языке Python удобно вернуть несколько значений в виде специальной структуры данных – кортежа.

2. После объявления первой функции создайте новую функцию, реализующую ввод имен игроков:

```
def Input_gamer():
    ig1 = input('Введите имя 1-го играющего ')
    ig2 = input('Введите имя 2-го играющего ')
    return (ig1, ig2)    # возвращается кортеж из двух переменных
```

3. Вместо существующего кода запроса имен игроков объявите две переменные и присвойте им результат вызова новой функции (кортеж с двумя переменными разложится в две новые переменные):

```
gamer1, gamer2 = Input_gamer()
```

Функциональность программы не изменилась, но теперь отдельный алгоритм ввода имен игроков вынесен в отдельную функцию.

Указание 5 (дополнительное). Разработка программы на основе классов

Другим вариантом построения программы является создание классов, описывающих предметную область.

Описание сущностей предметной области

Предметная область – это часть реального мира (в нашем случае предметная область определяется игрой), которая подлежит изучению с целью создания программных конструкций при реализации объектно-ориентированной парадигмы.

Сначала требуется разработать модель предметной области – определить основные сущности в терминах предметной области (эти сущности называются концептуальными классами).

1. Для выявления предметных сущностей (концептуальных классов) ответьте на следующие вопросы:
    a. Кто участвует в процессе (игре)?
        i. Ответ: игрок, кубик (хотя сам это и не подозревает 😊)

b. Как взаимодействуют между собой участники?
    i. Ответ: игрок бросает кубик (а не наоборот)
c. Кто управляет процессом игры, т.е. ведет подсчет очков и определяет победителя?
    i. Ответ: в нашем случае проверка после каждого броска и определение победителя происходит отдельно от игрока в главной программе.

2. Сформулируйте набор предметных сущностей и опишите взаимодействие между ними.

На основе ответов можно предложить следующую модель (набор концептуальных классов (обратите внимание – не программных классов, а именно концептуальных):

a. Игрок:
    i. Свойства: имя и число очков, полученных в сеансе игры
    ii. Поведение: бросать кубик
b. Кубик:
    i. Свойства: число очков
    ii. Поведение: переворачиваться (быть ‘брошенным’)
c. Игра:
    i. Свойства: не выявлено
    ii. Поведение: проверка одного сеанса (сравнение броска игроков), определение итогового победителя.
d. Отношения:
    i. Игрок реализует свое желание сделать ход посредством броска кубика
    ii. Игра определяет игроков, кубик и управляет всем процессом непосредственно игры, в итоге определяя победителя.

Это примерный перечень модели, может быть и другой результат – это определяется степенью погружения в проблему.

Реализация программных классов

Исходя из описания предметной области (концептуальных классов) можно предложить следующие решения по созданию программных классов:

- класс *Игрок* и класс *Кубик,*
- логику игры оставить в виде разработанных ранее функциях и в выражениях основной программы.

Предложенный вариант является простым решением, вообще можно создать отдельный класс *Игра*, но в текущем задании это не рассматривается.

1. Скопируйте файл прошлого решения в отдельный новый файл. Далее вы внесете изменения в существующее решение.
2. Откройте новый файл в редакторе ILDE.
3. В начале файла после инструкций импорта добавьте код, объявляющий два класса – *Игральная кость* и *Игрок*:

```
from random import randint
import time

# Игральная кость
class Dice:
    # конструктор, инициализирующий кубик при создании
    def __init__(self):
        self.n = randint(1, 6)

    # метод броска кубика – кубик “переворачивается”
    def roll(self):
        self.n = randint(1, 6)

# Игрок
class Gamer:
    # конструктор, инициализирующий игрока при создании
    # n – результат броска игрока
    def __init__(self, name, n = 0):
        self.name = name

    # метод броска игрока – “берем в руки” кубик
    # внутри вызываем реализацию метода броска кубика
    def dice_roll(self, dice):
        dice.roll()
        self.n = dice.n
        return self.n

    def __str__(self):
        return "Игрок {0.name}".format(self)
```

4. В функции, определяющую победителя измените имена переменных для вывода на имена объектов-игроков (они буду созданы ниже):

```
def who_champion(a, b):
    if a > b:
        return ('Выиграл {0}, число побед: {1}'.format(g1, a))
```

```
    elif a < b:
        return ('Выиграл {0}, число побед: {1}'.format(g2, b))
    else:
        return 'Ничья'
```

5. Функцию, реализующую ввод имен, а также ее вызов оставьте без изменения:

```
def Input_gamer():
    ig1 = input('Введите имя 1-го играющего ')
    ig2 = input('Введите имя 2-го играющего ')
    return (ig1, ig2)

gamer1, gamer2 = Input_gamer()
```

6. Сразу после этого кода вызова функции ввода имен создайте двух игроков – объекты-экземпляры класса `Gamer` с передачей в конструктор введенные имена:

```
g1 = Gamer(gamer1)
g2 = Gamer(gamer2)
```

7. Создайте объект – экземпляр класса игральной кости (кубика):

```
d1 = Dice();
```

8. Идея игры – бросания несколько раз кубика реализована в цикле `for..in`. При наличии созданных объектов (двух игроков и кубика) требуется ввести небольшие изменения в тело цикла:

```
for i in range(5):
    #Моделирование бросания кубика первым играющим
    print('Кубик бросает', g1)
    time.sleep(2)
    n1 = g1.dice_roll(d1)
    print('Выпало:', n1)

    #Моделирование бросания кубика вторым играющим
    print('Кубик бросает', g2)
    time.sleep(2)
    n2 = g2.dice_roll(d1)
    print('Выпало:', n2)

    #Определение результата (3 возможных варианта)
    if n1 > n2:
        print('В раунде выиграл', g1)
        k1 += 1
    elif n1 < n2:
        print('В раунде выиграл', g2)
```

```
            k2 += 1
        else:
            print('Ничья')
```

9. Завершение программы оставьте без изменений:

```
fin = who_champion(k1,k2)
print("Итог игры: ", fin)
```

Таким образом, программа содержит два класса, реализующие предметные сущности (игрок и кубик), функции, работающие с объектами класса и выражения, описывающие главную логику программы – непосредственно игру.

Литература

1. *Вигерс К.* Разработка требований к программному обеспечению/Пер. с англ. — М.: Издательство-торговый дом «Русская Редакция», 2014. — 736 с.
2. *Златопольский Д. М.* Основы программирования на языке Python. – М.: ДМК Пресс, 2017. – 284 с.
3. *Жуков Р.А.* Язык программирования Python: практикум : учеб, пособие М.: ИНФРА-М, 2019. – 216 с.
4. Как писать качественные пользовательские истории. [Электронный ресурс]. Режим доступа: https://netology.ru/blog/history-klient
5. Как писать User Story. [Электронный ресурс]. Режим доступа: https://medium.com/@alexandertvar/%D0%BA%D0%B0%D0%BA-%D0%BF%D0%B8%D1%81%D0%B0%D1%82%D1%8C-user-story-2410093b23c2